MobileComp: Real-Time Full Projector Compensation on Mobile Devices

Category: Research

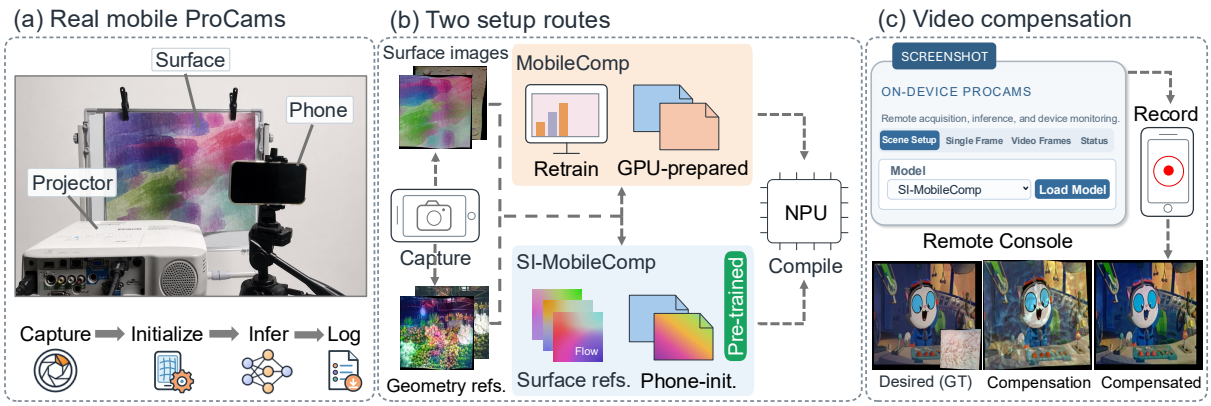


Figure 1: An integrated mobile workflow for real-time full projector compensation. **(a)** The smartphone closes the projector-camera systems (ProCams) loop by capturing setup observations, initializing or loading setup state, performing real-time inference, and recording results. **(b)** MobileComp deploys a GPU-prepared setup package, whereas SI-MobileComp constructs its setup package on the smartphone from geometric and surface references using shared pretrained weights. Both routes prepare compact graphs for neural processing unit (NPU) execution. **(c)** The remote console controls video compensation and displays the desired, compensation, and camera-captured frames while monitoring and recording the session.

Abstract

Full projector compensation aims to modify projector input image to compensate for geometric and photometric distortions introduced by a nonplanar, textured projection surface. Mobile deployment of this capability is highly desirable, as it provides user convenience and preserves data privacy by keeping projector inputs on-device, without uploading to external servers. Existing learning-based methods, however, typically require setup-specific training on a desktop-class GPU and computationally expensive inference, limiting self-contained mobile deployment. To solve this problem, we propose MobileComp, which, to the best of our knowledge, is the first framework to enable real-time full projector compensation on a smartphone. MobileComp performs real-time geometric correction and photometric compensation on the smartphone after preparing a compact compensation model on a desktop-class GPU. We further present Setup-Independent MobileComp (SI-MobileComp) to eliminates per-setup training using shared pre-trained weights. We design a compact student network, deployment-aligned quantization, and an asynchronous Android pipeline that overlaps frame preparation with inference on the smartphone-integrated neural processing unit (NPU). This pipeline integrates smartphone-based reference acquisition, setup initialization, video decoding, geometric correction, and NPU-based photometric compensation to form a complete mobile compensation system. Evaluations across diverse setups show that MobileComp achieves real-time video compensation on a smartphone, outperforming desktop-oriented baselines that run at less than 1 FPS on the same device with a speedup of over 60x. We will release the source code and data upon the publication of the work.

Index Terms: Full projector compensation, projector-camera systems, mobile computing, neural processing units, spatial augmented reality

1 Introduction

Projection mapping and spatial augmented reality display digital content on real-world surfaces primarily using Projector-Camera Systems (ProCams) [15, 24]. When projecting onto nonplanar and textured surfaces, the setup configuration causes geometric distortion, while surface reflectance, ambient illumination, and device responses produce spatially varying photometric distortion. Full projector compensation aims to modify a projector input image so that the camera-captured projection result matches the desired appearance [1, 19].

Enabling such compensation on mobile devices is highly desirable for practical deployment. Portable ProCams allow users to set up projection interactively in everyday environments without relying on stationary infrastructure or external servers. Moreover, on-device processing preserves data privacy: the media content being compensated remains locally on the user's smartphone, rather than being transmitted to the cloud for processing. However, this convenience and privacy come at a cost. Mobile hardware imposes stringent limits on compute, memory, and power.

Despite this clear need, existing approaches fall short of mobile-ready full projector compensation. Portable ProCams integrate projection and sensing into a compact unit, reducing reliance on external devices [5]. Self-configuring projectors estimate display geometry from captured observations and reduce repeated manual calibration [45]. In parallel, learning-based full projector compensation methods have been improved for real-time inference [3]. However, these advances address sensing and calibration effort or network inference latency separately; they do not jointly support smartphone-side setup initialization and real-time full compensation. Existing compensation pipelines consequently leave setup-dependent geometry, photometry, or model preparation tied to external computation or entangled with per-frame processing. Within a fixed video session, the ProCams configuration remains invariant across frames; repeatedly estimating this setup state consumes the real-time budget, whereas retraining after each setup change limits portability. Moreover, efficient mobile projector compensation depends not only on parameter count and isolated network latency, but also on the exported operator graph, tensor interfaces, video decoding, image warping, memory transfer, and pipeline scheduling.

To address these constraints, we present *MobileComp*, which constructs a setup-specific sampling grid and compact photometric compensation network once on a desktop-class GPU from smartphone-captured data. It then reuses this setup package throughout the video

session, restricting the per-frame path to fixed-grid geometric correction and photometric compensation. Building on MobileComp, we further present *Setup-Independent MobileComp (SI-MobileComp)*, which follows SIComp's unseen-setup formulation and keeps shared pretrained weights fixed across target setups [30]. For a new setup, the system acquires one geometric reference pair and five surface references, from which the smartphone initializes a sampling grid and multi-scale surface-feature cache without per-setup GPU retraining or network-weight updates. The resulting setup package is reused across subsequent video frames, while the shared network remains fixed across unseen setups.

Both frameworks require their photometric compensation networks to fit the computation, memory, and operator constraints of the target smartphone. We therefore construct compact student networks and optimize them through teacher-student distillation, per-channel quantization-aware training (QAT), and quantize-dequantize (QDQ) export. For MobileComp, we reduce the channel widths of the photometric compensation network to construct a compact dense student. For SI-MobileComp, the SIComp teacher [30] contains attention, normalization, softmax, and matrix-multiplication operators that fragment delegation to the Qualcomm AI Engine Direct (QNN) backend. We replace them with QNN-friendly convolution, ReLU, addition/subtraction, bilinear resizing, and clipping to construct a fully convolutional student. Because SI-MobileComp separates surface encoding from real-time compensation, we first calibrate the quantized surface graph and then use its outputs to calibrate the downstream real-time graph, preserving their deployed tensor interface. At run time, MediaCodec decodes each video frame, Java Native Interface (JNI) kernels apply the cached sampling grid for geometric correction, and LiteRT/QNN performs photometric compensation on the smartphone-integrated neural processing unit (NPU) through the Hexagon Tensor Processor (HTP). Asynchronous preprocessing overlaps frame conversion and warping with QNN execution, while the application records outputs and runtime statistics locally. Fig. 1 summarizes this complete mobile workflow, from smartphone-based setup acquisition and initialization to real-time full compensation and recording.

Across eight target setups, MobileComp and SI-MobileComp achieve camera-captured compensation quality comparable with the desktop-oriented baselines. On a Xiaomi 13 Pro, they reach 72.30 and 64.76 frames per second (FPS), respectively, whereas the evaluated desktop-oriented baselines remain below 1 FPS on the same device. These results demonstrate that our mobile compensation framework enables real-time full projector compensation while maintaining comparable compensation quality. Our key contributions are summarized below:

- To the best of our knowledge, we present the first mobile framework for full projector compensation. It integrates smartphone-based reference acquisition and setup initialization with real-time geometric correction and photometric compensation.
- We introduce two full projector compensation frameworks for mobile devices. MobileComp separates one-time setup preparation from real-time smartphone processing, while SI-MobileComp further eliminates per-setup GPU retraining. Together, they reduce setup dependence and computational overhead, enabling efficient end-to-end compensation on mobile hardware.
- MobileComp and SI-MobileComp achieve camera-captured compensation quality comparable to that of the desktop-oriented baseline, while enabling real-time, full projector compensation on mobile devices at 72.30 and 64.76 FPS, respectively, over 60x faster than the baseline.

2 Related Work

Projector-camera systems couple geometric and photometric transformations between the projector input and camera-captured result [1]. Full projector compensation jointly inverts these transformations to reproduce a desired appearance on nonplanar textured surfaces [19]. We review projector compensation, adaptation across setups and viewpoints, and efficient mobile inference.

2.1 Projector compensation methods

Classical methods explicitly model projector-camera geometry, device radiometry, and light transport. Geometric calibration estimates projector-camera correspondence [35], while radiometric models characterize device responses, surface reflectance, and environmental illumination [38, 13]. Recent formulations incorporate spectral and perceptual objectives [34], jointly estimate geometric and radiometric parameters [47, 14, 32], or model direct and global illumination, compressive light transport, and interreflection [61, 37, 42, 53]. These methods depend on accurate geometric and radiometric calibration, which must be repeated when the projector-camera configuration changes.

Preparation effort can be reduced through online adaptation, self-configuration, and mobile capture. Prior systems update geometry and radiometry during operation [9, 40], infer display geometry from captured observations [45], or use handheld cameras for geometric and color calibration [62, 51]. Modular monocular ProCams preserve pixel correspondence under focus and zoom changes [63]; multi-projector systems support unknown surfaces or improve photometric consistency [56, 44]; and embedded-camera targets reduce the projection–capture cycles required for dense-array calibration [29]. These systems simplify acquisition or calibration but still reconstruct setup-specific state after a configuration change.

Learning-based methods approximate the nonlinear geometric and photometric inverse from captured data. CompenNet predicts photometric compensation from desired and surface images [17], whereas CompenNeSt++ jointly learns a geometric sampling grid and photometric compensation network [19]. DeProCams decomposes ProCams image formation for relighting, compensation, and shape reconstruction [18]. CompenHR targets high-resolution full compensation through attention-based grid refinement and pixel shuffle/unshuffle operations [57], while CompenRT performs low-resolution compensation followed by learned upsampling [3]. While learning-based methods eliminate the need for additional devices and system geometry or radiometry calibration, their backends often depend on desktop-oriented computing hardware, which limits direct deployment on mobile devices.

Differentiable alternatives optimize projector inputs through projection simulation [41], physics-based decomposition [33], or differentiable multi-bounce path tracing [31]. Related formulations incorporate lighting-model initialization and diffusion priors [52, 59]. Dynamic projector deblurring estimates defocus and luminance attenuation from consecutive frames [27] or combines optical-flow-based correction with a lightweight network trained on synthetic data [28]. These methods address dynamic geometry and defocus rather than mobile full photometric compensation on textured surfaces.

2.2 Projector compensation across setups and viewpoints

A projector-camera setup changes with the device pose, projection surface, or photometric conditions. Dynamic projection mapping maintains registration through surface tracking [36, 26], depth- or infrared-based fitting [23], and automated recalibration after device movement [21]. High-speed face tracking with a lens-shift co-axial configuration reduces temporal and optical misalignment [43], while temporally coded illumination with an event camera establishes correspondence under bright ambient light [10]. Multi-projector

systems optimize luminance, non-rigid registration, and seamless rendering over dynamic surfaces [49, 50, 22]. Dense synthetic-aperture arrays suppress occlusion shadows [39], and lighting-aware systems mitigate environmental illumination through heterogeneous projectors and mixed or target-excluding light fields [54, 64, 11]. Together, these approaches improve robustness to geometric, temporal, occlusion, and illumination changes, but generally rely on continuous sensing, specialized hardware, or repeated setup-specific estimation.

These systems estimate or update setup-specific geometry, photometry, or lighting during operation. ViComp adapts its photometric model online from projected and captured video frames [58], whereas SIComp learns shared weights across diverse setups and combines online optical flow with intensity-varying surface priors to compensate unseen setups without fine-tuning [30]. Setup independence differs from viewpoint independence: Neural Projection Mapping reconstructs a neural reflectance field from multi-view captures [8], while GS-ProCams estimates a Gaussian scene representation for efficient view-agnostic ProCams simulation and compensation [6]. Although these methods enable view-agnostic operation, mobile full projector compensation additionally requires smartphone-side setup adaptation and real-time processing within on-device computation and memory budgets.

2.3 Efficient mobile inference

Efficient mobile inference combines compression, lightweight architectures, and hardware-aware computation. Model-centric techniques reduce storage through pruning and trained quantization [16], employ inverted residuals or compound scaling [46, 55], and enable integer-only execution through quantization-aware training [25]. Once-for-All further selects hardware-specialized subnetworks through progressive shrinking [2]. Beyond parameter reduction, Deep Bilateral Learning shifts image enhancement to a low-resolution bilateral grid [12], MobileNeRF replaces ray marching with mobile rasterization [4], and Mobile-GS removes depth sorting while compressing Gaussian attributes [7]. Together, these methods show that parameter reduction alone is insufficient for mobile deployment; efficient execution also requires model architectures and execution paths designed for the target hardware.

Together, prior work exposes three challenges for mobile full projector compensation. First, existing compensation models remain computationally demanding and contain operators that execute inefficiently on mobile accelerators. Second, the complete video-compensation pipeline must satisfy stringent end-to-end latency requirements while sustaining real-time throughput. Third, a mobile system should adapt across setups without repeated recalibration or model retraining, yet existing setup-independent networks remain too complex for efficient mobile execution.

3 Method

3.1 Preliminary

A full projector compensation setup consists of a projector-camera pair and a nonplanar textured projection surface at a fixed pose. Following the formulation of Huang et al. [19], we decompose its project-and-capture process into a photometric transformation $\mathcal{H}_s$ and a geometric transformation $\mathcal{G}_s$, where the subscript s denotes setup-dependent quantities. Given desired frame x'_T, full compensation predicts projector input $x^*_{s,T}$ as

$$x^*_{s,T} = \mathcal{H}_s^{\dagger}\left(\mathcal{W}_s(x'_T), \rho_s\right), \tag{1}$$

where $\mathcal{W}_s = \mathcal{G}_s^{-1}$ warps the desired frame to the projector canonical domain, ρ_s denotes the aligned camera-captured surface observations, and $\mathcal{H}_s^{\dagger}$ compensates the photometric distortion. The camera-captured projection is $\widetilde{x}^*_{s,T} = \mathcal{G}_s(\mathcal{H}_s(x^*_{s,T}))$. This decomposition separates reusable setup state from per-frame compensation, providing the basis for both mobile deployment protocols.

3.2 Projector compensation on mobile devices

Deploying full projector compensation on a smartphone requires more than reducing network size. The system must isolate reusable setup computation from real-time frame processing, map the compensation networks to accelerator-supported operators, and preserve the tensor distributions exchanged by separately exported setup-time and per-frame graphs. We address these requirements through setup-time/per-frame decomposition, teacher-student model adaptation, and deployment-aligned quantization. The on-device implementation in Sec. 3.3 then integrates the resulting setup packages and mobile graphs with native geometric correction and asynchronous Android execution. This sequence directly implements the reusable setup state, mobile-adapted compensation networks, and complete smartphone path introduced in Sec. 1.

3.2.1 Setup-time and per-frame decomposition

The projector-camera geometry and surface response remain fixed during a video session. We therefore represent them as reusable setup state and keep their estimation outside the real-time path. We refer to each deployment configuration as a *target setup*. MobileComp prepares a sampling grid and photometric compensation network for each target setup on a desktop-class GPU. Building on this decomposition, SI-MobileComp shares pre-trained weights across target setups and reconstructs only the sampling grid and surface features on the smartphone. As summarized in Fig. 2, both frameworks reuse their setup package across subsequent frames, leaving only geometric correction and photometric compensation in the per-frame path.

Both protocols express their sampling grids in a common projector canonical domain; Supplementary Fig. S1 details the FOV identification and affine normalization used to construct this domain.

MobileComp adopts CompenHR's GANet-PANet decomposition [57]. Because geometry and surface response remain fixed within a setup, we export GANet's sampling grid G_s and evaluate PANet's surface branch once during setup preparation. We absorb the resulting surface features into the setup-specific state of the compact photometric compensation network f_{θ_s}. SI-MobileComp instead keeps the network weights shared across setups and stores only the smartphone-initialized sampling grid Ω_s and multi-scale surface-feature cache $\mathcal{F}_s$. We represent the two reusable setup packages as

$$\lambda_1 = \{G_s, f_{\theta_s}\}, \qquad \lambda_2 = \{\Omega_s, \mathcal{F}_s\}. \tag{2}$$

Here, λ_1 and λ_2 denote setup packages rather than scalar loss weights or tunable hyperparameters. To expose their shared geometric operation, let Γ_s denote either G_s or Ω_s. For each output pixel p, the native warp $\mathcal{W}(I;\Gamma_s)$ samples the continuous source position $\Gamma_s(p)$ by bilinear interpolation. We precompute the four neighboring indices and their interpolation weights once per setup, eliminating grid construction and geometry estimation from the per-frame path. This operator instantiates the backward warp $\mathcal{W}_s$ in Eq. (1); its detailed form is provided in the supplementary material.

Using λ_1, MobileComp predicts the projector input for desired frame x'_T, denoted $x^{*,\mathrm{MC}}_{s,T}$, as

$$x^{*,\mathrm{MC}}_{s,T} = f_{\theta_s}\left(\mathcal{W}(x'_T; G_s)\right). \tag{3}$$

To remove the per-setup GPU training required to construct λ_1, we further develop SI-MobileComp.

SI-MobileComp follows SIComp's setup-independent formulation [30] and keeps the pre-trained weights fixed across target setups. For a target setup, the projector displays a geometric reference

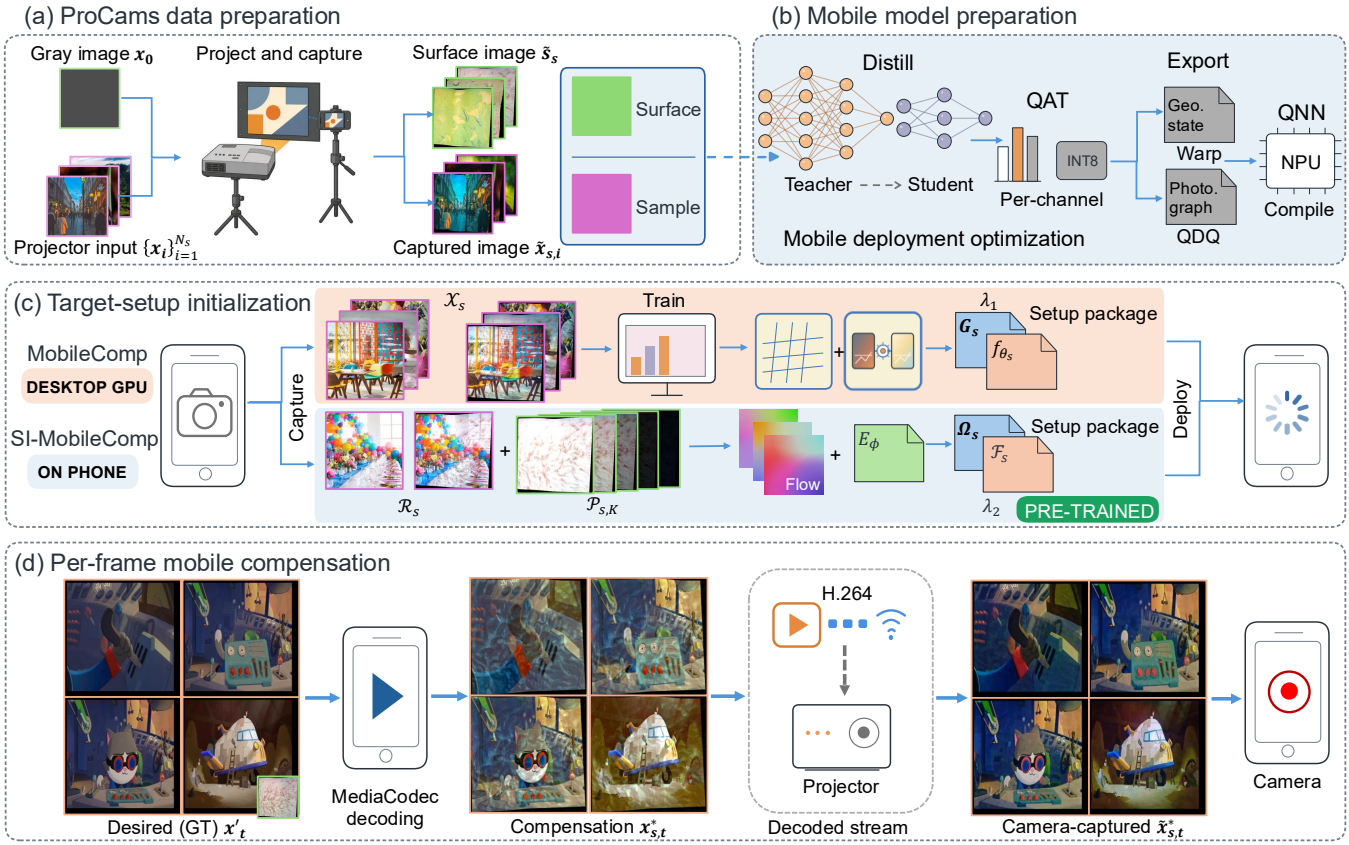


Figure 2: Mobile full projector compensation workflow. **(a)** Data acquisition from a ProCams setup records gray and surface references together with paired projector-camera images. **(b)** Distillation and per-channel QAT produce compact models, while export separates reusable geometry from NPU-ready photometric graphs. **(c)** MobileComp prepares setup state on a desktop-class GPU; SI-MobileComp initializes it on the smartphone using shared pre-trained weights. **(d)** The smartphone decodes video, performs real-time full compensation, streams the H.264 output to the projector, and records the camera-captured result.

r^p, and the smartphone captures r^c_s, forming $\mathcal{R}_s = \{r^p, r^c_s\}$. The smartphone also captures five intensity-varying surface references, denoted by $\mathcal{P}_{s,K} = \{r^c_{s,k}\}_{k\in K}$, where K is the set of projector intensity indices and k indexes one intensity level. FlowFormer [20] estimates the backward flow once on the smartphone CPU and produces the normalized sampling grid Ω_s without target-setup network optimization or GPU retraining.

To encode the spatially varying surface response, we align and concatenate $\mathcal{P}_{s,K}$ using the same grid:

$$\widetilde{S}_s = \mathrm{Concat}_{k\in K}\, \mathcal{W}(r^c_{s,k}; \Omega_s). \tag{4}$$

The surface encoder E_ϕ processes the resulting 15-channel tensor once and constructs a multi-scale feature cache:

$$\mathcal{F}_s = \{F^1_s, F^2_s, F^3_s\} = E_\phi(\widetilde{S}_s). \tag{5}$$

The cache persists throughout the session, and its memory grows with the channel widths and spatial dimensions of the three feature scales; the exact tensor-memory accounting is given in the supplementary material. During per-frame inference, the native warp computes $\mathcal{W}(x'_T; \Omega_s)$, and the shared network R_θ combines the warped desired frame with the cached surface features:

$$x^{*,\mathrm{SI}}_{s,T} = R_\theta\left(\mathcal{W}(x'_T; \Omega_s), F^1_s, F^2_s, F^3_s\right). \tag{6}$$

Together, the sampling grid and feature cache instantiate the smartphone-initialized package λ_2 in Eq. (2). When the setup changes, SI-MobileComp reconstructs λ_2 while keeping θ fixed; neither FlowFormer nor E_ϕ enters the per-frame path. This decomposition moves setup initialization to the smartphone and preserves the same real-time sequence of geometric correction followed by photometric compensation.

3.2.2 Teacher-student model adaptation

After setup-dependent computation is removed from the real-time path, the remaining compensation networks must still fit the computation, memory, and operator constraints of the target smartphone. Unstructured pruning reduces parameter count but retains wide intermediate tensors and sparse channel layouts that do not match the dense accelerator path. We therefore construct compact dense students for both deployment protocols and use frozen teachers to preserve compensation behavior during structural adaptation.

For MobileComp, we reconstruct PANet [57] as a uniformly narrower dense network and transfer channels consistently across convolutions, skip connections, attention blocks, and transposed convolutions. The 50%-width student reduces the channel configuration from $(48, 64, 128, 256)$ to $(24, 32, 64, 128)$ and the PANet parameter count from 867,200 to 223,472. We keep GANet frozen so that structural adaptation changes only the photometric compensation network. Let $\hat{x}_{S,s,i}$ and $\hat{x}_{T,s,i}$ denote the student and frozen FP32 CompenHR teacher predictions for training pair i in setup s. We optimize the student with

$$\mathcal{L}_{\mathrm{MC}} = w_{\mathrm{task}}\mathcal{L}_{\mathrm{task}} + w_{\mathrm{out}}\mathcal{L}_{\mathrm{out}} + w_{\mathrm{feat}}\mathcal{L}_{\mathrm{feat}} + w_{\mathrm{color}}\mathcal{L}_{\mathrm{color}}, \tag{7}$$

where the task term combines supervised pixel and structural objectives [57, 60]. Output distillation transfers the teacher prediction,

while feature distillation matches normalized channel-energy maps at selected PANet stages despite unequal teacher and student widths. The CIEDE2000 term [48] further constrains perceptual color error. We schedule w_{task} and w_{out} to emphasize teacher guidance early in training and increase the supervised objective as the student stabilizes. The supplementary material specifies the component losses, selected feature stages, coefficients, and schedule.

For SI-MobileComp, structural adaptation must preserve SIComp's setup-time/per-frame interface while removing operators that impede mobile delegation. The SIComp teacher [30] uses Swin blocks and convolutional-block attention, whose normalization, softmax, and matrix-multiplication operations increase activation traffic and fragment the target delegation path. We replace this operator mix with convolution, ReLU, addition/subtraction, bilinear resizing, and clipping, producing a fully convolutional student that maps directly to QNN-supported operations.

The 255,705-parameter student separates a 73,483-parameter surface encoder from a 182,222-parameter real-time branch. At full, half, and quarter scales, the encoder extracts 3-, 24-, and 48-channel surface features; the real-time branch subtracts them from the corresponding desired-frame features and processes the result through two 48-channel residual blocks, bilinear resizing, and additive skip connections before a final convolution clips the compensation frame to $[0,1]$. This fully convolutional design preserves the reusable surface interface and maps both exported graphs to the target accelerator path.

We adapt the shared SI-MobileComp student using examples from *source setups*, i.e., projector-camera configurations reserved for training. To preserve both compensation outputs and setup-dependent surface features, we split the frozen SIComp $\text{SCNet}_{\text{surf5}}$ teacher [30] at the same setup-time/per-frame boundary. For each source setup, we precompute the FlowFormer grid, warped reference stack, and teacher surface features. Following surrogate inverse learning [19, 30], we feed each camera capture $\widetilde{x}_{s,i} = \mathcal{G}_s(\mathcal{H}_s(x_i))$ through the learned inverse and supervise it with the paired projector input x_i. Teacher-only desired frames provide additional output and feature targets without requiring projector-domain labels.

Let $\mathcal{B}$ denote a mixed batch of paired and teacher-only samples, and let $\mathcal{B}_{\text{p}} \subseteq \mathcal{B}$ contain its paired samples. We optimize the SI-MobileComp student with

$$\mathcal{L}_{\text{SI}} = \mathcal{L}_{\text{sup}}(\mathcal{B}_{\text{p}}) + \mathcal{L}_{\text{out}}(\mathcal{B}) + 0.25\mathcal{L}_{\text{surf}}(\mathcal{B}), \tag{8}$$

where $\mathcal{L}_{\text{sup}}$ combines paired-pixel and SSIM objectives [60], $\mathcal{L}_{\text{out}}$ distills compensation predictions over the complete mixed batch, and $\mathcal{L}_{\text{surf}}$ aligns the three scales of cached surface features. We define $\mathcal{L}_{\text{sup}} = 0$ when $\mathcal{B}_{\text{p}}$ is empty, avoiding undefined normalization and nonexistent projector-domain targets. Training-only 1×1 heads align unequal student and teacher channel widths and are discarded after adaptation. The supplementary material gives the expanded batch objective, feature heads, and exact coefficients.

3.2.3 Deployment-aligned quantization

After structural adaptation, integer conversion must preserve the tensor interfaces and feature distributions of the exported graphs. For MobileComp, per-channel QAT adapts the compact PANet to integer arithmetic, after which we compile its PANet-only ONNX graph into a TensorFlow Lite model for LiteRT/QNN execution on HTP. The graph executes internally in INT8 while retaining FP32 NHWC input and NCHW output; our mobile measurements therefore include the required layout and type conversions.

For SI-MobileComp, we apply QAT [25] through PyTorch FX, using the QNNPACK configuration for activation fake quantization and per-output-channel convolution-weight quantization. This configuration simulates integer conversion during training; QNN/HTP remains the on-device execution backend. Before ONNX export, we copy the QAT-optimized convolution parameters into clean FP32 modules because training observers are not runtime operators. Subsequent QDQ calibration determines the activation ranges and inserts the quantization boundaries used by the exported graphs.

The surface and real-time graphs form a directed interface: the latter consumes the quantized surface features produced by the former. Calibrating them independently with clean features would therefore introduce a training-deployment mismatch. We reproduce the deployed dependency by calibrating the surface graph first and using its QDQ outputs to calibrate the real-time graph. Let $\mathcal{D}_{\text{surf}}^{\text{cal}}$ contain the warped surface stacks from the balanced source-only calibration split, and let z_j denote calibration image j associated with setup s_j. The sequential procedure is

$$\begin{aligned} Q_{\text{surf}} &= \text{Calibrate}\Big(E_\phi; \mathcal{D}_{\text{surf}}^{\text{cal}}\Big), \\ Q_{\text{rt}} &= \text{Calibrate}\Big(R_\theta; \Big\{\Big(\mathcal{W}(z_j;\Omega_{s_j}), Q_{\text{surf}}(\widetilde{S}_{s_j})\Big)\Big\}_j\Big). \end{aligned} \tag{9}$$

Consequently, the real-time graph observes the three QDQ surface outputs that it receives at deployment rather than clean FP32 features. Both graphs use MinMax calibration, unsigned 8-bit activations, signed per-channel 8-bit weights, and convolutional QDQ nodes. Qualcomm AI Hub compiles Q_{surf} and Q_{rt} into TensorFlow Lite models, which LiteRT executes through QNN on HTP. The external interfaces remain FP32 NCHW, and the complete mobile measurements therefore include layout conversion, quantization boundaries, and feature transfer.

3.3 On-device implementation

Accelerating the photometric network alone does not determine video throughput, because decoding, color conversion, geometric warping, tensor transfer, and inference all remain on the smartphone path. Moreover, the target QNN path does not expose the setup-dependent grid warp as a fully delegated operator. We therefore implement the bilinear warp defined above as a native kernel and reserve HTP execution for photometric compensation. For each setup, the kernel precomputes the four source indices and weights of every output pixel. JNI kernels then write directly into reusable NCHW or NHWC buffers, avoiding repeated grid construction and intermediate allocation.

During SI-MobileComp initialization, the application stores Ω_s, warps $\mathcal{P}_{s,K}$, executes the LiteRT surface encoder, and serializes $\mathcal{F}_s$. This sequence reconstructs the complete setup package before video processing begins. The supplementary material describes the state validation and serialization checks used to prevent incomplete packages from entering the real-time profile.

For each video frame, MediaCodec decodes the MP4 input, the CPU converts YUV420 to ARGB and applies native geometric correction, and LiteRT/QNN performs photometric compensation on HTP. The application can record the compensation output and runtime statistics locally. Serial execution leaves the CPU or NPU idle between dependent stages. Inspired by cross-frame parallel processing in video compensation [58], we instead overlap independent work across adjacent frames. A pull-based decoder applies backpressure, while a bounded queue and double buffering prepare frame $T+1$ concurrently with QNN inference for frame T.

Let τ_{dec}, τ_{conv}, and τ_{warp} denote the decoding, color-conversion, and geometric-warp times, respectively, and let τ_{QNN} denote the QNN inference time. We model the frame-preparation time and the steady-state frame intervals as

$$\begin{aligned} \tau_{\text{prep}} &= \tau_{\text{dec}} + \tau_{\text{conv}} + \tau_{\text{warp}}, \\ I_{\text{sync}} &= \tau_{\text{prep}} + \tau_{\text{QNN}}, \qquad I_{\text{async}} = \max\big(\tau_{\text{prep}}, \tau_{\text{QNN}}\big) + \delta_{\text{sched}}. \end{aligned} \tag{10}$$

where all times are measured in milliseconds and δ_{sched} accounts for queue synchronization, scheduling, and output work outside the

overlapped stages. The model captures the steady-state interval when CPU preparation overlaps NPU inference, and Sec. 4.4 evaluates the resulting scheduling gain.

End-to-end latency spans the path from frame admission to post-processing, whereas I_{async} measures the steady-state interval between completed frames. We measure wall-clock FPS directly and record per-stage and end-to-end latency, dropped frames, and queue occupancy; Supplementary Fig. S2 shows the corresponding controls and counters.

Together, reusable setup packages, native geometric correction, QNN photometric graphs, and asynchronous scheduling form the complete mobile path, keeping setup initialization outside per-frame processing.

4 Experiments

We evaluate camera-captured compensation quality and complete smartphone execution for both deployment protocols, then isolate structured compression and quantization, teacher-student supervision, and asynchronous scheduling. Physical projection-recapture measurements are reported separately from model-domain ablations.

4.1 Experimental setup

Hardware and image domains. Our ProCams setup uses an EPSON CB-965 projector and the 50-megapixel Sony IMX989 main camera of a Xiaomi 13 Pro smartphone. Model training was performed on an NVIDIA A100 SXM GPU, and the trained models were exported on a Mac equipped with an Apple M4 system-on-chip (SoC). On-device evaluation was conducted on the same smartphone, powered by the Qualcomm Snapdragon 8 Gen 2 Mobile Platform and running Android 15. The projector input patterns were generated at 600×600 pixels. Camera images were captured and supplied to the models at 960×540 pixels, whereas the models produced 600×600 projector images.

Data splits and comparison methods. We split the data by projector-camera setup. Two source setups, which provide data for adapting the shared SI-MobileComp student, contain 1,000 supervised projector-camera image pairs and 400 balanced teacher-only images. A disjoint setup used only for validation and model selection supplies 200 validation pairs and remains excluded from training and QDQ calibration. Eight target setups are held out for the main evaluation until the architecture, checkpoint, quantization policy, and runtime are frozen.

We compare MobileComp and SI-MobileComp with CompenNeSt++ [19], CompenHR [57], and CompenRT [3]. MobileComp and the three baselines train a separate model for each target setup. SI-MobileComp instead keeps its shared pre-trained weights fixed across all target setups and initializes only λ_2 on the smartphone, without model retraining. Camera-captured quality uses matched desired content and identical projector-camera conditions within each setup. For smartphone timing, our methods execute their complete LiteRT/QNN paths, whereas each baseline runs its original graph on the same device; the comparison therefore measures end-to-end target-device deployability.

Training and export. MobileComp optimizes a setup-specific compact model for each target setup. In contrast, SI-MobileComp trains a shared model once on the source setups and keeps its weights fixed during target-setup deployment. We train this shared model with AdamW for 2,500 FP32 steps followed by 500 QAT steps, using respective learning rates of 2×10^{-4} and 5×10^{-5} and weight decays of 10^{-4} and 10^{-5}. Both stages use batch size 4 and 256×256 crops, and graph export uses ONNX opset 18.

Evaluation protocols. For camera-captured quality, we project each compensation frame onto the surface and capture the resulting projection with the smartphone. We report PSNR and SSIM [60] (higher is better), together with RMSE, CIEDE2000 ΔE [48], and LPIPS-VGG [65] (lower is better). These complementary metrics cover signal-level reconstruction error, structural similarity, color difference, and perceptual distance. We first average frames within each setup and then weight the eight target setups equally. Model-domain ablations instead compare predicted projector images directly with their common projector-domain targets without projecting and recapturing the outputs.

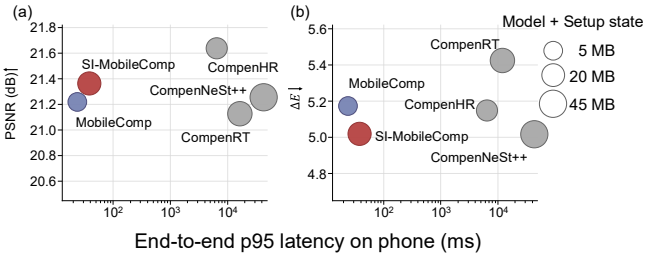


Figure 3: Quality-efficiency comparison on the Xiaomi 13 Pro using camera-captured results. **(a)** PSNR and **(b)** CIEDE2000 ΔE are plotted against p95 end-to-end latency; marker area encodes the combined size of the model and reusable setup state. MobileComp and SI-MobileComp retain compensation quality comparable to the desktop-oriented baselines while reducing latency to the real-time range.

Mobile measurements use batch one. After warm-up, eight setup-specific sessions measure model execution, end-to-end latency from frame admission through postprocessing, wall-clock FPS, model-file size, and peak device memory. The default SI-MobileComp mode enables asynchronous preprocessing, sustained-performance mode, and HTP execution; because adjacent stages overlap, we measure FPS directly.

4.2 Overall comparison

We first evaluate whether the mobile designs maintain camera-captured compensation quality while satisfying real-time smartphone throughput. We report the physical projection-recapture quality and complete target-device performance under the matched protocol aforementioned in Tab. 1, and visualize the quality-efficiency trade-offs in Fig. 3.

Across the eight target setups, SI-MobileComp obtains the highest SSIM and the second-best value for each of the other four quality metrics using the same pre-trained weights. MobileComp and SI-MobileComp reach 72.30 and 64.76 FPS, respectively, whereas all three desktop-oriented baselines remain below 1 FPS on the same smartphone. Thus, both mobile paths achieve real-time full projector compensation while retaining competitive camera-captured quality.

The qualitative comparisons in Fig. 4 mirror the quantitative findings. MobileComp and SI-MobileComp recover fine text and texture details comparably to the desktop baselines, while the uncompensated projections remain severely distorted by surface texture and reflectance.

4.3 Setup reuse and on-device performance

We measure SI-MobileComp initialization from reference upload through setup-package serialization, excluding reference acquisition and device positioning. Geometry estimation runs with ONNX Runtime on the CPU, whereas the surface and real-time graphs execute through LiteRT/QNN on HTP.

Fig. 5 and Tab. 1 characterize the complete smartphone path. SI-MobileComp reconstructs its setup package in 29.70 ± 1.95 s without per-setup GPU retraining, including 18.35 s for geometry estimation and grid construction. Once initialized, MobileComp and SI-MobileComp reach 72.30 and 64.76 FPS with p95 end-to-end latencies of 17.66 and 27.87 ms, respectively. SI-MobileComp uses a 0.22 MB model and a 17.28 MB persistent float32 surface cache; its 561.44 MB peak RAM further includes intermediate tensors, media buffers, and runtime workspace. Asynchronous scheduling

Table 1: Overall camera-captured quality and target-smartphone deployment. Quality is averaged equally across eight setups. MobileComp and SI-MobileComp retain competitive quality while reaching real-time throughput, whereas the desktop-oriented baselines remain below 1 FPS on the same Xiaomi 13 Pro. Size denotes model-file size, E2E Lat. reports p50/p95 latency, Exec. denotes model execution time, and RAM denotes peak memory; bold marks the best mean.

	Camera-captured compensation quality					Target-smartphone deployment				
Method	PSNR↑	RMSE↓	SSIM↑	ΔE ↓	LPIPS↓	Size (MB)↓	FPS↑	E2E Lat. (ms)↓	Exec. (ms)↓	RAM (MB)↓
CompenHR	**21.6630**	**0.0868**	0.7982	5.1432	0.1775	12.87	< 1	4529.07/5074.88	4472.32	**219.99**
CompenNeSt++	21.2133	0.0913	0.7902	**5.0057**	**0.1585**	45.00	< 1	33229.56/33695.64	33250.48	338.45
CompenRT	21.1273	0.0914	0.7894	5.4027	0.1745	27.06	< 1	12800.35/12890.01	12682.71	293.57
MobileComp (Ours)	21.2943	0.0906	0.7890	5.1561	0.1757	1.07	**72.30**	**16.51/17.66**	**4.64**	264.25
SI-MobileComp (Ours)	21.3722	0.0879	**0.8020**	5.0933	0.1628	**0.22**	64.76	25.53/27.87	14.69	561.44

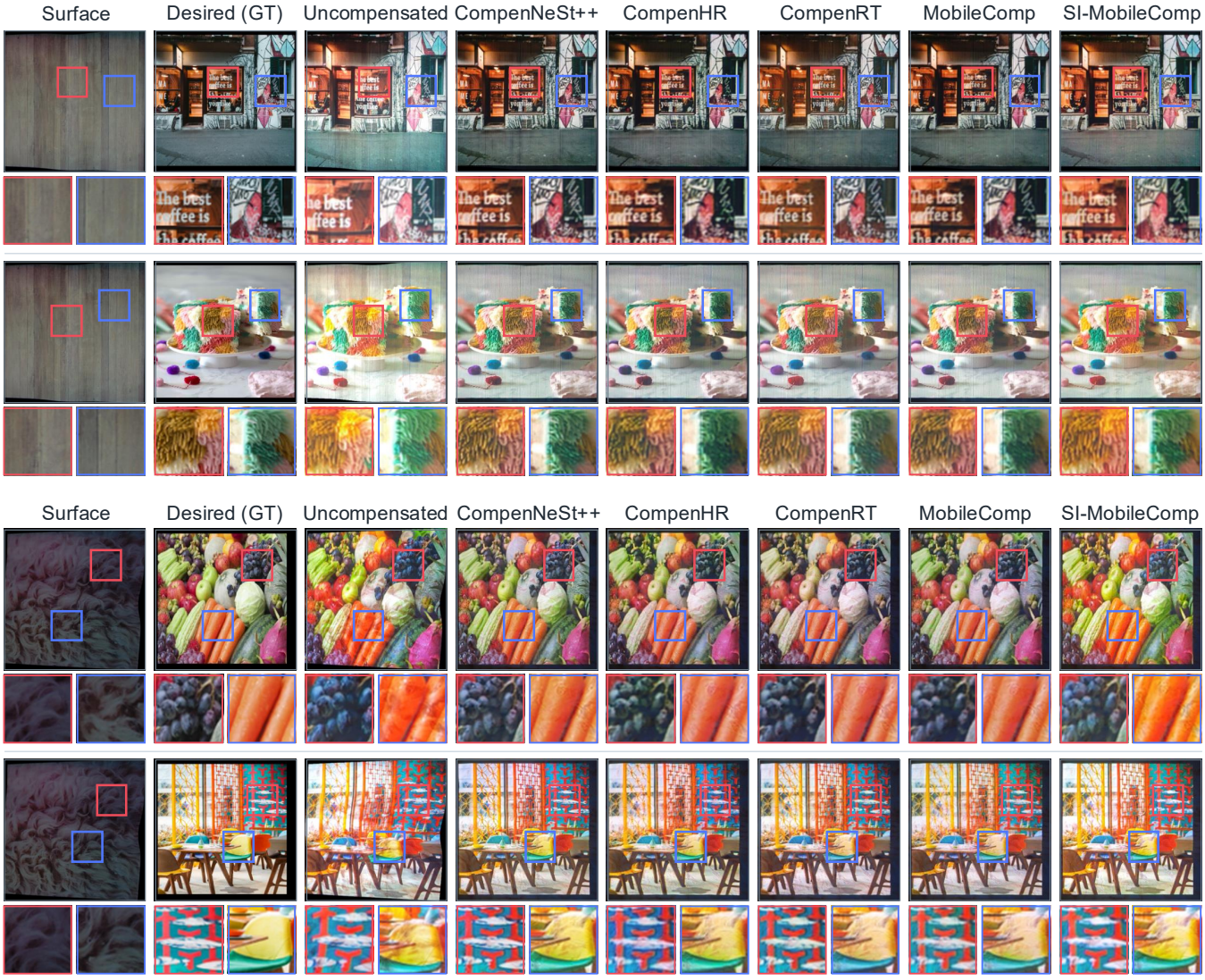


Figure 4: Camera-captured full projector compensation results across four desired contents and two projection surfaces. Columns show the surface image, desired frame, uncompensated camera capture, and five compensation results; colored boxes mark enlarged regions. Compared with the uncompensated captures, the compensation results reduce surface-dependent color and intensity distortion under the same physical imaging conditions. The supplementary material provides video compensation frames from additional scenes, while the accompanying demo video shows the complete video compensation results.

overlaps CPU preparation with NPU inference, preserving real-time throughput while setup-dependent computation remains outside the per-frame path.

Both frameworks reload their serialized setup packages after an application restart, avoiding repeated reference acquisition and setup reconstruction. Tab. 5 shows that MobileComp and SI-MobileComp restore their runtime state in 1551.15 and 5071.49 ms, respectively, before returning to steady-state throughputs of 72.30 and 64.76 FPS.

4.4 Ablation studies

We isolate structured compression, quantization, teacher-student supervision, and asynchronous scheduling to measure their effects on mobile quality and efficiency under otherwise fixed configurations.

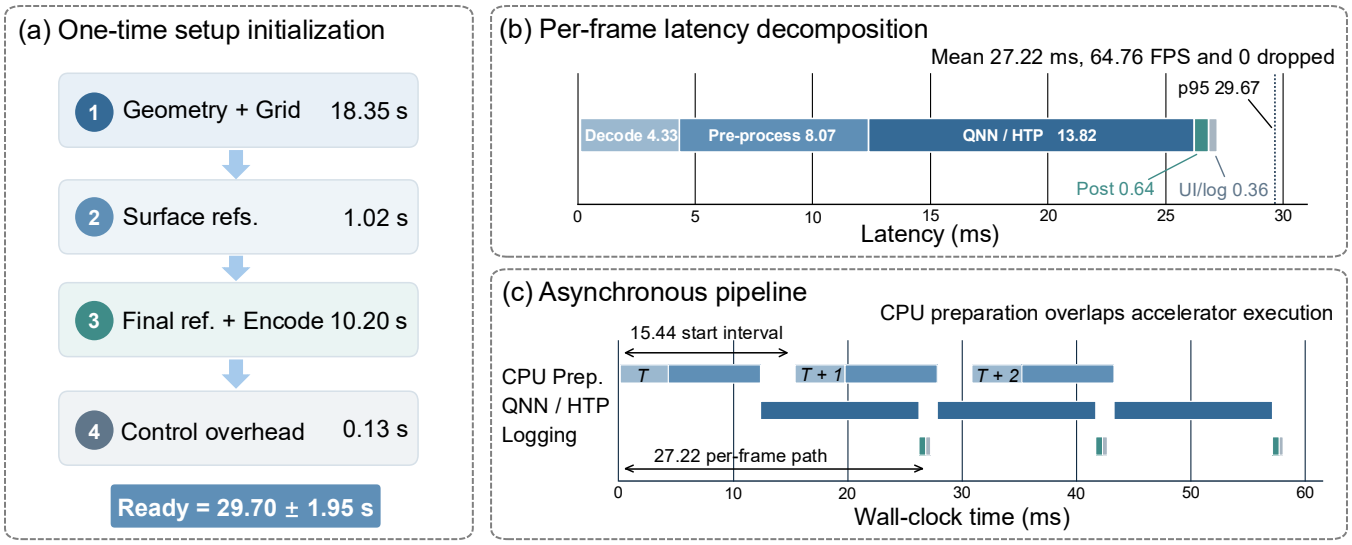


Figure 5: Smartphone-side setup initialization and per-frame runtime. **(a)** One-time SI-MobileComp initialization averaged across eight setups, excluding reference capture. **(b)** Mean stage latency, p95 latency, throughput, and drops from eight 300-frame runs. **(c)** CPU preparation overlaps QNN/HTP execution, so wall-clock throughput is not the reciprocal of per-frame latency.

MobileComp compression and quantization. To evaluate how MobileComp balances compensation quality and mobile efficiency, we trace the uncompressed FP32 CompenHR teacher through structured pruning and distillation to the final QAT/QDQ INT8 model. All variants share the sampling grid, inputs, Android pipeline, and target smartphone.

Under the shared protocol reported in Tab. 2, structured pruning reduces PANet from 1,327,646 to 223,472 parameters. Distillation then improves all five camera-captured metrics over fine-tuning, including PSNR from 19.2315 to 19.5429 dB. Converting the distilled model with QAT/QDQ reduces p95 latency from 68.70 to 17.66 ms and raises throughput from 23.30 to 72.30 FPS. These results show that distillation recovers quality after structural compression, while integer deployment provides the principal runtime gain.

Teacher-student supervision. To evaluate the supervision strategy used to adapt the setup-independent student, we ablate output and feature distillation while fixing the 24/48-channel architecture, data split, and training schedule. The three variants disable both terms, output distillation, or feature distillation; active supervision uses the same frozen SIComp checkpoint [30]. We export every variant as matched INT8 QDQ surface and real-time graphs, execute them through LiteRT/QNN, and compare their projector-domain outputs with common targets across six setups.

SI-MobileComp obtains the best mean on all five model-domain metrics under the fixed architecture and deployment settings (Tab. 3). Removing feature distillation reduces PSNR from 20.5673 to 19.4398 dB and increases ΔE from 5.4296 to 6.4648; removing output distillation or both terms causes larger degradation. This controlled comparison confirms the joint contribution of output and feature supervision.

The qualitative comparison further corroborates these trends: SI-MobileComp retains local contrast and reduces ΔE over broad image regions (Fig. 6), supporting the joint contribution of output and feature distillation.

Asynchronous frame scheduling. Finally, we evaluate the contribution of asynchronous scheduling to mobile throughput. SI-MobileComp overlaps CPU frame preparation with QNN inference, enabling CPU and NPU execution to proceed concurrently.

With the two stages overlapped, scheduling increases throughput from 45.74 to 64.76 FPS, a 41.6% gain, and reduces the frame interval from 21.86 to 15.44 ms while maintaining comparable QNN execution and p95 latency, as reported in Tab. 4. This controlled result confirms that CPU–NPU overlap converts pipeline idle time into sustained throughput.

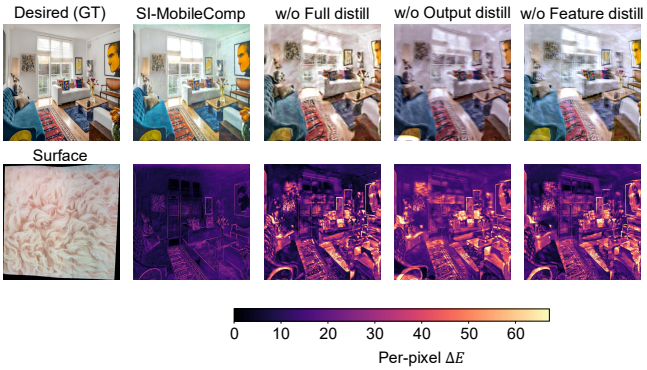


Figure 6: Qualitative teacher-student ablation corresponding to Tab. 3. The upper row compares the desired target, SI-MobileComp, and three distillation ablations; the lower row shows the surface and per-pixel ΔE maps on a shared scale.

5 Discussion And Limitations

The results show that mobile full projector compensation requires complete pipeline co-design rather than parameter reduction alone. By separating reusable setup state from per-frame processing, MobileComp and SI-MobileComp maintain camera-captured quality comparable with desktop-oriented methods while sustaining real-time smartphone execution. MobileComp prioritizes throughput through GPU-prepared setup state, whereas SI-MobileComp extends the workflow to new setups without per-setup GPU retraining or network-weight updates.

The controlled ablations explain these operating points. Distillation recovers quality after structural compression, QAT/QDQ enables efficient integer execution, and asynchronous CPU/NPU scheduling improves throughput by 41.6%. The measured footprint further shows that deployment must account for cached setup features, intermediate tensors, media buffers, and runtime workspace in addition to network weights. Together, these results establish full-system co-design as the mechanism behind real-time mobile compensation.

The reusable setup formulation assumes a fixed projector-camera

Table 2: MobileComp compression and quantization ablation. Structured pruning (Struct.) constructs the compact student, distillation (Distill.) recovers compensation quality, and INT8 QAT enables real-time QNN execution. Quality is averaged across six setups and runtime across four 300-frame sessions; bold marks the best result.

Method	Struct. pruning	Distill.	INT8 QAT	Params↓	Model (MB)↓	PSNR↑	RMSE↓	SSIM↑	ΔE ↓	LPIPS↓	E2E p95 (ms)↓	FPS↑
CompenHR	–	–	–	1,327,646	12.87	**20.0306**	**0.1026**	**0.7651**	**5.7693**	**0.1880**	5074.88	< 1
MobileComp (Prun+Fine-tune)	✓	–	–	**223,472**	3.96	19.2315	0.1121	0.7399	6.1106	0.1938	67.60	22.40
MobileComp (Prun+Distill)	✓	✓	–	**223,472**	3.96	19.5429	0.1081	0.7536	5.9318	0.1906	68.70	23.30
MobileComp (Ours)	✓	✓	✓	**223,472**	**3.33**	19.6264	0.1072	0.7540	5.8084	0.1885	**17.66**	**72.30**

Table 3: Teacher-student ablation for SI-MobileComp. Model-domain quality is averaged equally across six setups. Removing output or feature distillation degrades all five metrics, supporting both supervision signals. In the Method column, out. and feat. denote output and feature; bold marks the best mean.

Method	PSNR↑	RMSE↓	SSIM↑	ΔE ↓	LPIPS↓
SI-MobileComp w/o distill	17.8160	0.1311	0.7331	7.2078	0.2234
SI-MobileComp w/o out. distill	17.8758	0.1299	0.7436	7.9552	0.2333
SI-MobileComp w/o feat. distill	19.4398	0.1089	0.7583	6.4648	0.1941
SI-MobileComp (Ours)	**20.5673**	**0.0959**	**0.7830**	**5.4296**	**0.1699**

Table 4: Efficiency gain from asynchronous frame scheduling in SI-MobileComp. Results are averaged over eight 300-frame sessions on the Xiaomi 13 Pro; Int. denotes the measured frame interval, and bold marks the better result in each metric.

Scheduling	QNN (ms)↓	E2E p95 (ms)↓	Int. (ms)↓	FPS↑	Gain↑
Asynchronous	13.82	29.67	**15.44**	**64.76**	**+41.6%**
Synchronous	**13.14**	**27.91**	21.86	45.74	—

Table 5: Cold-start recovery and steady-state performance on the Xiaomi 13 Pro. Load restores the serialized method-specific setup package—MobileComp's sampling grid and setup-specific model, or SI-MobileComp's sampling grid and surface-feature cache—together with the deployment assets, and initializes the corresponding runtime context. Cold-start rows characterize setup-state reuse after an application restart, whereas steady-state rows isolate sustained real-time execution with the setup package and runtime context resident. "–" denotes that loading is bypassed in steady state. Values are means over ten setup-specific sessions; steady-state measurements follow 30 untimed frames and then time 300 frames.

Method	State	Load (ms)↓	First E2E (ms)↓	QNN (ms)↓	E2E p95 (ms)↓	FPS↑
MobileComp	Cold	1551.15	46.42	4.80	19.32	71.06
MobileComp	Steady	–	36.20	4.64	17.66	72.30
SI-MobileComp	Cold	5071.49	57.35	14.25	29.49	64.06
SI-MobileComp	Steady	–	47.10	14.69	27.87	64.76

configuration and canonical viewpoint within each session. Configuration changes trigger setup-state reconstruction, while continuously moving devices would require incremental geometry and surface-feature updates. Dynamic surfaces, persistent occlusion, transparent materials, and view-agnostic operation would further require temporal or multi-view representations. Future work will extend these capabilities while reducing initialization and memory costs and evaluating additional mobile hardware.

6 Conclusion

We presented MobileComp and SI-MobileComp, two successive frameworks that separate reusable setup state from real-time smartphone processing. MobileComp reuses a GPU-prepared sampling grid and compact compensation network, while SI-MobileComp further replaces per-setup GPU retraining with smartphone-side reconstruction using fixed shared weights. Compact students, deployment-aligned quantization, and asynchronous execution enable MobileComp and SI-MobileComp to retain camera-captured quality comparable with desktop-oriented baselines while reaching 72.30 and 64.76 FPS, respectively, on a Xiaomi 13 Pro. These results demonstrate that co-designing setup computation, deployed networks, and the media-inference pipeline enables real-time full projector compensation on a smartphone.

References

[1] O. Bimber, D. Iwai, G. Wetzstein, and A. Grundhöfer. The visual computing of projector-camera systems. *Comput. Graph. Forum*, 27(8):2219–2245, 2008. 1, 2

[2] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-all: Train one network and specialize it for efficient deployment. In *ICLR*, 2020. 3

[3] M. Chen and B. Huang. Real-time high-resolution projector compensation. In *IEEE ICME*, pages 1–6, 2025. 1, 2, 6

[4] Z. Chen, T. Funkhouser, P. Hedman, and A. Tagliasacchi. MobileNeRF: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *CVPR*, pages 16569–16578, 2023. 3

[5] G. Cortes, E. Marchand, G. Brincin, and A. Lécuyer. MoSART: Mobile spatial augmented reality for 3D interaction with tangible objects. *Front. Robot. AI*, 5:93, 2018. 1

[6] Q. Deng, J. Li, H. Ling, and B. Huang. GS-ProCams: Gaussian Splatting-based projector-camera systems. *IEEE TVCG*, 31(11):9845–9855, 2025. 3

[7] X. Du, Y. Wang, K. Zhan, and X. Yu. Mobile-GS: Real-time Gaussian Splatting for mobile devices. In *ICLR*, 2026. 3

[8] Y. Erel, D. Iwai, and A. H. Bermano. Neural projection mapping using reflectance fields. *IEEE TVCG*, 29(11):4339–4349, 2023. 3

[9] K. Fujii, M. D. Grossberg, and S. K. Nayar. A projector-camera system with real-time photometric adaptation for dynamic environments. In *CVPR*, volume 1, pages 814–821, 2005. 2

[10] Y. Fujimoto, T. Sawabe, M. Kanbara, and H. Kato. Structured light of flickering patterns having different frequencies for a projector-event-camera system. In *IEEE VR*, pages 582–588, 2022. 2

[11] K. Fujimura, H. Kusuyama, M. Takeuchi, and D. Iwai. High-contrast projection mapping under light field illumination with LED display and aperiodic lens array. *IEEE TVCG*, 32(5):4250–4259, 2026. 3

[12] M. Gharbi, J. Chen, J. T. Barron, S. W. Hasinoff, and F. Durand. Deep bilateral learning for real-time image enhancement. *ACM TOG*, 36(4):118:1–118:12, 2017. 3

[13] M. D. Grossberg, H. Peri, S. K. Nayar, and P. N. Belhumeur. Making one object look like another: Controlling appearance using a projector-camera system. In *CVPR*, volume 1, pages 452–459, 2004. 2

[14] A. Grundhöfer and D. Iwai. Robust, error-tolerant photometric projector compensation. *IEEE TIP*, 24(12):5086–5099, 2015. 2

[15] A. Grundhöfer and D. Iwai. Recent advances in projection mapping algorithms, hardware and applications. *Comput. Graph. Forum*, 37(2):653–675, 2018. 1

[16] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *ICLR*, 2016. 3

[17] B. Huang and H. Ling. End-to-end projector photometric compensation. In *CVPR*, pages 6810–6819, 2019. 2

[18] B. Huang and H. Ling. DeProCams: Simultaneous relighting, compensation and shape reconstruction for projector-camera systems. *IEEE TVCG*, 27(5):2725–2735, 2021. 2

[19] B. Huang, T. Sun, and H. Ling. End-to-end full projector compensation. *IEEE TPAMI*, 44(6):2953–2967, 2022. 1, 2, 3, 5, 6

[20] Z. Huang, X. Shi, C. Zhang, Q. Wang, K. C. Cheung, H. Qin, J. Dai, and H. Li. FlowFormer: A transformer architecture for optical flow. In *ECCV*, pages 668–685, 2022. 4

[21] M. T. Ibrahim, M. Gopi, and A. Majumder. Self-calibrating dynamic projection mapping system for dynamic, deformable surfaces with jitter

correction and occlusion handling. In *IEEE ISMAR*, pages 293–302, 2023. 2

[22] M. T. Ibrahim, M. Gopi, and A. Majumder. Real-time seamless multi-projector displays on deformable surfaces. *IEEE TVCG*, 30(5):2527–2537, 2024. 3

[23] M. T. Ibrahim, G. Meenakshisundaram, and A. Majumder. Dynamic projection mapping of deformable stretchable materials. In *VRST*, pages 1–5, 2020. 2

[24] D. Iwai. Projection mapping technologies: A review of current trends and future directions. *Proc. Jpn. Acad., Ser. B*, 100(3):234–251, 2024. 1

[25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR*, pages 2704–2713, 2018. 3, 5

[26] S. Kagami and K. Hashimoto. Animated stickies: Fast video projection mapping onto a markerless plane through a direct closed-loop alignment. *IEEE TVCG*, 25(11):3094–3104, 2019. 2

[27] Y. Kageyama, D. Iwai, and K. Sato. Online projector deblurring using a convolutional neural network. *IEEE TVCG*, 28(5):2223–2233, 2022. 2

[28] Y. Kageyama, D. Iwai, and K. Sato. Efficient distortion-free neural projector deblurring in dynamic projection mapping. *IEEE TVCG*, 30(12):7544–7557, 2024. 2

[29] T. Kawano, K. Miura, and D. Iwai. Breaking the scalability limit of multi-projector calibration with embedded cameras. In *CVPR*, pages 21573–21582, 2026. 2

[30] H. Li, Q. Deng, J. Li, H. Ling, and B. Huang. Setup-independent full projector compensation. *IEEE TVCG*, 32(5):3189–3199, 2026. 2, 3, 5, 8

[31] J. Li, Q. Deng, H. Ling, and B. Huang. DPCS: Path tracing-based differentiable projector-camera systems. *IEEE TVCG*, 31(5):3666–3676, 2025. 2

[32] Y. Li, A. Majumder, M. Gopi, C. Wang, and J. Zhao. Practical radiometric compensation for projection display on textured surfaces using a multidimensional model. *Comput. Graph. Forum*, 37(2):365–375, 2018. 2

[33] Y. Li, W. Yin, J. Li, and X. Xie. Physics-based efficient full projector compensation using only natural images. *IEEE TVCG*, 30(8):4968–4982, 2024. 2

[34] Y. Maeda and D. Iwai. Projector radiometric compensation using a 2D spectroradiometer. *Opt. Express*, 34(9):15979–15993, 2026. 2

[35] D. Moreno and G. Taubin. Simple, accurate, and robust projector-camera calibration. In *3DIMPVT*, pages 464–471, 2012. 2

[36] G. Narita, Y. Watanabe, and M. Ishikawa. Dynamic projection mapping onto deforming non-rigid surface using deformable dot cluster marker. *IEEE TVCG*, 23(3):1235–1248, 2017. 2

[37] S. K. Nayar, G. Krishnan, M. D. Grossberg, and R. Raskar. Fast separation of direct and global components of a scene using high frequency illumination. *ACM TOG*, 25(3):935–944, 2006. 2

[38] S. K. Nayar, H. Peri, M. D. Grossberg, and P. N. Belhumeur. A projection system with radiometric compensation for screen imperfections. In *ICCV Workshop on Projector-Camera Systems (ProCams)*, pages 1–8, 2003. 2

[39] T. Okamoto, M. Takeuchi, M. Sawayama, and D. Iwai. Shadowless projection mapping for tabletop workspaces with synthetic aperture projector. *IEEE TVCG*, 32(5):3033–3043, 2026. 3

[40] H. Park, M.-H. Lee, B.-K. Seo, J.-I. Park, M.-S. Jeong, T.-S. Park, Y. Lee, and S.-R. Kim. Simultaneous geometric and radiometric adaptation to dynamic surfaces with a mobile projector-camera system. *IEEE TCSVT*, 18(1):110–115, 2008. 2

[41] J. Park, D. Jung, and B. Moon. Projector compensation framework using differentiable rendering. *IEEE Access*, 10:44461–44470, 2022. 2

[42] P. Peers, D. K. Mahajan, B. Lamond, A. Ghosh, W. Matusik, R. Ramamoorthi, and P. Debevec. Compressive light transport sensing. *ACM TOG*, 28(1):3:1–3:18, 2009. 2

[43] H.-L. Peng, K. Sato, S. Nakagawa, and Y. Watanabe. Perceptually-aligned dynamic facial projection mapping by high-speed face-tracking method and lens-shift co-axial setup. *IEEE TVCG*, 31(10):6824–6838, 2025. 2

[44] P. Pjanic, S. Willi, D. Iwai, and A. Grundhöfer. Seamless multi-projection revisited. *IEEE TVCG*, 24(11):2963–2973, 2018. 2

[45] R. Raskar, J. van Baar, P. Beardsley, T. Willwacher, S. Rao, and C. Forlines. iLamps: Geometrically aware and self-configuring projectors. *ACM TOG*, 22(3):809–818, 2003. 1, 2

[46] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *CVPR*, pages 4510–4520, 2018. 3

[47] M. Shahpaski, L. R. Sapaico, G. Chevassus, and S. Susstrunk. Simultaneous geometric and radiometric calibration of a projector-camera pair. In *CVPR*, pages 4885–4893, 2017. 2

[48] G. Sharma, W. Wu, and E. N. Dalal. The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color Res. Appl.*, 30(1):21–30, 2005. 5, 6

[49] C. Siegl, M. Colaianni, L. Thies, J. Thies, M. Zollhöfer, S. Izadi, M. Stamminger, and F. Bauer. Real-time pixel luminance optimization for dynamic multi-projection mapping. *ACM TOG*, 34(6):1–11, 2015. 3

[50] C. Siegl, V. Lange, M. Stamminger, F. Bauer, and J. Thies. FaceForge: Markerless non-rigid face multi-projection mapping. *IEEE TVCG*, 23(11):2440–2446, 2017. 3

[51] C.-H. Son and Y.-H. Ha. Color correction of images projected on a colored screen for mobile beam projector. *J. Imaging Sci. Technol.*, 52(3):030505, 2008. 2

[52] W. Sun, G. Noh, K. Ko, and B. Moon. Lighting model-guided initialization for gradient descent-based projector compensation. *IEEE Access*, 13:163607–163617, 2025. 2

[53] S. Takeda, D. Iwai, and K. Sato. Inter-reflection compensation of immersive projection display by spatio-temporal screen reflectance modulation. *IEEE TVCG*, 22(4):1424–1431, 2016. 2

[54] M. Takeuchi, H. Kusuyama, D. Iwai, and K. Sato. Projection mapping under environmental lighting by replacing room lights with heterogeneous projectors. *IEEE TVCG*, 30(5):2151–2161, 2024. 3

[55] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, pages 6105–6114, 2019. 3

[56] J.-P. Tardif, S. Roy, and M. Trudeau. Multi-projectors for arbitrary surfaces without explicit calibration nor reconstruction. In *3DIM*, pages 217–224, 2003. 2

[57] Y. Wang, H. Ling, and B. Huang. CompenHR: Efficient full compensation for high-resolution projector. In *IEEE VR*, pages 135–145, 2023. 2, 3, 4, 6

[58] Y. Wang, H. Ling, and B. Huang. ViComp: Video compensation for projector-camera systems. *IEEE TVCG*, 30(5):2347–2356, 2024. 3, 5

[59] Y. Wang, H. Ling, and B. Huang. DiffPC: Diffusion-based projector photometric compensation. *IEEE TVCG*, 32(5):3104–3113, 2026. 2

[60] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE TIP*, 13(4):600–612, 2004. 4, 5, 6

[61] G. Wetzstein and O. Bimber. Radiometric compensation through inverse light transport. In *Pacific Graphics*, pages 391–399, 2007. 2

[62] C. Xie, H. Shishido, Y. Kameda, and I. Kitahara. Geometric calibration of projector using a mobile camera for spatial augmented reality. *Trans. Virtual Reality Soc. Jpn.*, 25(2):138–147, 2020. 2

[63] K. Yamamoto, D. Iwai, I. Tani, and K. Sato. A monocular projector-camera system using modular architecture. *IEEE TVCG*, 29(12):5586–5592, 2023. 2

[64] M. Yasui, R. Iwataki, M. Ishikawa, and Y. Watanabe. Projection mapping with a brightly lit surrounding using a mixed light field approach. *IEEE TVCG*, 30(5):2217–2227, 2024. 3

[65] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, pages 586–595, 2018. 6